CORE RDK PROGRAM

RDKB High Level API

Defining the Core: RDK-B (Broadband) & RDK-V (Video/Entertainment) Specification Baseline

Version	0.1 Draft — for review
Date	14 August 2026
Owner	Charles Moreman, RDKM
Approval Authority	?
Classification	Confidential — Internal
Scope	RDKB High Level API Specification
Target Completion	30 September 2026

Executive Summary

The RDKB Broadband High-Level API Specification documents the high-level API used in RDK Broadband. This is sometimes referred to as the “North Bound Interface”.

This high-level API is used by components (including downloaded Apps) to communicate with other components (including downloaded Apps).

More specifically, this document defines high-level inter-process communication (IPC) for the following use cases:

- Communication between a broadband app and software components packaged in the monolith image
- Communication between a broadband app and other broadband apps
- Communications between a software component packaged in the monolith image and other software components packaged in the monolith image.

Table of Contents

1. Objective & Scope

1.1 Objective

Define, document, technically review, and prepare for governance endorsement of the RDKB High-Level API definition. This document forms part of the Core RDK specification baseline — the set of components, interfaces and hardware requirements that together define "the Core" — across both the Broadband and Video/Entertainment profiles, establishing the foundation on which Phases 2–4 (operationalization, capability evolution and enforcement) will build.

1.2 In-Scope for this Document

The scope of this document includes definitions and specifications for:

- High-level Operations: A definition of high-level operations supported by this API.
- Data Model Structure: An overall description of the structure of data model elements / objects used in this high-level API.
- Data Model Documentation Template: A template used to define documentation standards for data model elements / objects. This template is used when creating Component Interface Definitions. This template model contains sufficient details to define the "contract" between various IPC endpoints that consume or provide these data model elements / objects. This template model includes specific fields that must be used in documentation and specifies the details of a machine-readable documentation format.
- Interface Inventory and API Coverage Report: A list of component interface definition documents used in RDKB and the status of each of these documents.

1.3 Out of Scope (Addressed in Other Documents)

- While the interface inventory list is defined in this document, these documents themselves are out of scope. Each of these interface documents provides a full description of the data model elements and objects supported by that interface. Each interface definition document is separately versioned and mapped to one or more versions of a specific RDKB component. The component interface definition may also be subject to feature flags that control how the component is built. In this case, the interface definition will document any applicable feature flags and how these feature flags affect the interface. Component Interface Definitions will follow the Data Model Documentation template defined in this document. Each component interface definition document will be stored within the github repo containing the source code and documentation for the RDKB open-source component providing that interface.
- Future Dashboard tooling is out of scope for this document. This dashboard tooling will support the creation of data model subsets filtered to match only the interfaces provided by the list of versioned components and feature flags used in a specific software build. This dashboard tooling will also support creation of a summary list that describes all data model elements / objects that are supported in the sum of all components across multiple RDKB component versions and feature flags.
- The scope of this document does not include definitions of the low-level API specs. Low-level API specs define how to send and receive messages containing the high-level API operations and data model elements / objects. Low-level API specs contain definitions of bindings used for one or more software languages or may define a language independent API. Low-level API specs contain information on low-

level operations, error codes, and other infrastructure and details necessary to support the high-level API. RBUS API documentation provides an example of a commonly used low-level API spec. The RBUS API for C language implementations is defined by the "rbus.h" file in the RBUS repo located in github.

- Definitions used for the older IPC approach known as "SysEvent" are not in scope for this document. This is used for legacy code in some components. It does not follow a standard and is planned to be deprecated as schedules permit. This older IPC approach is not planned to be supported for downloaded Apps.

2. Reference Documents

[TR-181] The TR-181 specification is an evolving spec maintained by the Broadband Forum (BBF). This spec is published at BBF – TR-181 – Device Data Model for CWMP Endpoints and USP Agents

[TR-106] The TR-106 specification defines the data model template used by TR-181. While this spec contains useful information, the documentation model used in RDKB supersedes this spec and provides additional information not covered in TR-106. The TR-106spec is published at Technical Library | Broadband Forum

3. Definitions

Software Component: Software source code and documentation used to build executable code that provides a logical grouping of similar features. A component is contained in one or more github repos. A component repo branch is versioned.

RDKB Core Component: An RDKB software component controlled and maintained by the RDK community and open-sourced on github. RDKB Core Components include a Component Interface Definition as part of the documentation contained in the github repo.

Monolith Image: This is software packaged in a "main" image and used to upgrade the device firmware. It may be "double banked" to support a running image and a backup image used if the running image gets corrupted or fails to boot successfully. The Monolith Image typically contains many software components that are packaged and tested together.

Broadband App: A software component with its executable code packaged separately from software components in the monolith image. Components packaged as Broadband Apps operate like software components in the monolith image except that:

- App lifecycles may be independently controlled separately from the lifecycle of the monolith image. Apps may be downloaded/installed, started, stopped, upgraded, and un-installed separately.
- Apps may optionally run in a container that limits what the app can access. It can be important to limit the ability of an app to affect other services, limit the scope of regression testing and provide increased security especially for semi-trusted apps from 3rd party companies.
- (future) Apps may optionally contain metadata that controls an ACL allow list to limit which data model elements / objects can be provided by an App and/or controls an ACL allow list that limits which data model elements / objects can be consumed by the App. This ACL filtering may be used to define a

subset of the full high level API definition that each specific App may access. This metadata may be provided by the service provider, system integrator or app provider.

- A software component packaged as a Broadband App has the promise of reduced test scope, shorter deployment schedules and reduced risk compared to an upgrade of the device's monolith image.

Packaging Flexibility: Build recipes may be created to alternately package a software component as a Broadband App or package the same software component as part of a monolith image.

Component Interface Definition: This is a high-level data model interface definition supported by a specific component. The component interface definition uses a well-defined machine-readable format to describe the set of data model elements / objects provided by an RDKB component (including downloadable Apps). This definition may include feature flags that may be used to optionally include or exclude parts of the data model provided by the component. These feature flags are controlled at build-time. The Component Interface Definition is versioned and mapped to one or more versions of the applicable software component.

Data Model Consumer and Provider Components: Software components (including components packaged as broadband Apps) may both consume and provide data model elements/objects. A software component may "consume" data models provided by other software components. A software component may also "provide" data model elements / objects that are consumed by other software components. A component that provides data model elements / objects will register these data models using a low-level system bus API and respond to messages from other components that need to consume these data models. The implementer of a component that provides data models is responsible for the code that implements the features / services controlled by these data models and/or responsible for providing an interface to a south-bound HAL API to access these features/services.

Data Model Element: A data model element is a named entity at the lowest levels of the TR-181 hierarchy. An element is a generic term that may be implemented as a "Parameter" (property), an "Event" or a Method (command).

Data Model Object: A data model object is a named entity that defines a portion of the data model hierarchy. It may be at the top level (called the root object), at an intermediate level or may define a multi-instance object that works like an array. Objects have other objects or elements below them in the data model hierarchy.

4. High-level API Operations

The RDK Broadband High-Level API defines a set of supported operations that may be applied to specific data model element / object names.
The list of supported operations below must be supported by any low-level API used to transport messages containing high-level API data model elements / objects. The low-level API may also provide other infrastructure operations not described here.

Note: This document describes the high-level API operations. A low-level API that provides software bindings and message transport must also be supported. The definition of the low-level API is NOT in scope for this document.

Supported operations include:

- **GET**
- **SET**
- **ADD INSTANCE**
- **REMOVE INSTANCE**
- **SUBSCRIBE ON CHANGE**
- **SUBSCRIBE ON INTERVAL**
- **NOTIFY**
- **INVOKE SYNCHRONOUS**
- **INVOKE ASYNCHRONOUS**

4.1 GET Operation

This operation may be used to retrieve the current value(s) corresponding to a supported data model element / object name. The supported name may represent either an “element” representing a single value, or it may represent an “object” containing multiple property values.

This operation is optional for specific named data model elements / objects. Software components that provide named data model elements / objects may or may not support this operation on each named data model element / object. Support for specific operations is documented in the component interface definition for each component.

The GET operation

- This operation begins when a GET message is sent from a data model component. This message requests the current value(s) associated with the named data model element / object.
- The system routes the GET message to the single component that provides the name (if any). If a single component providing the entire data model element/object name cannot be found, a message containing the appropriate error is sent back to the requesting consumer component.
- If a single provider component was found, the GET message is routed to the provider component. The provider of the data model element checks to confirm that the GET operation is supported on this data model element / object. If not supported, a message containing the appropriate error is sent by the provider component and the system routes this error message to the requesting data model consumer component. If the GET operation is supported by the provider component, the

provider retrieves the value(s) and sends a message back to the system containing the element / object name and value(s).

- Note: Depending on the provider component implementation, the software may use a HAL interface to retrieve the value(s) from lower-level hardware dependent software or it may retrieve the value from memory. Data model elements used for real time status (example: WiFi Received Signal Strength) will typically retrieve this value from a HAL API. Data model elements used for a configured property (example: configured WiFi SSID name) will typically retrieve this value from a memory location.
- The system routes this message containing the data model name and value(s) to the original requesting data model consumer component

4.2 SET Operation

This operation may be used to request a write of a supported data model element/object name to a value(s). The supported name may represent either a single “property” value or it may represent a “object” containing multiple property values.

This operation is optional for specific named data model elements / objects. Software components that provide named data model elements / objects may or may not support this operation on each named data model element / object. Support for specific operations is documented in the component interface definition for each component.

The SET operation:

- This operation begins when a SET message is sent from a data model consumer component. This message identifies the name of a data model element / object and requests that the value(s) be changed to the value(s) included in this message.
- The system routes the SET message to the single component that provides the data model element /object (if any). If a single component providing the entire data model element/object cannot be found, a message containing the appropriate error is sent back to the requesting consumer component.
- If a single provider component was found, the SET message is routed to the corresponding provider component. The provider component checks to confirm that the SET operation is supported on this data model element / object. The provider may optionally check if the requested new value(s) is within a range of supported values. If not supported or the value is not within a range of supported values, a message containing the appropriate error is sent by the provider component and the system routes this error message back to the requesting data model consumer component. If the SET operation is supported by the provider component and the value(s) is within a range of supported values, the provider stores the value(s) and sends a message back to the system to confirm the operation was successful.
- The system routes this confirmation message back to the original requesting data model consumer component.

4.3 ADD INSTANCE Operation

This operation may be used to add a new instance to a supported named multi-instance data model object.

This operation is optional for specific named data model objects. Software components that provide named data model objects may or may not support this operation on each named data model object. Support for specific operations is documented in the component interface definition for each component.

Note: Depending on the component interface definition corresponding to the provider component implementation, the number of instances supported by some multi-instance objects may be static or dynamic. Multi-instance objects with a fixed number of instances are said to be “static” multi-instance objects. The ADD INSTANCE operation is not supported for static multi-instance objects. An example of a static multi-instance object is an object that has an instance for each WiFi radio in a device. Since the number of WiFi radios is determined by the hardware implementation, the number of instances cannot be changed by other components dynamically. Multi-instance objects that support ADD INSTANCE are said to be “dynamic” multi-instance objects.

The ADD INSTANCE operation

- This operation begins when an ADD INSTANCE message is sent from a data model consumer component. This message requests that a new instance be added to a named multi-instance data model object. The message may optionally include an “alias” name for the new instance. The alias name may be used to refer to the instance in the future. If an alias name is not supplied, the instance may be referred to using an instance number.
- The system routes the ADD INSTANCE message to the single component that provides the object name (if any). If a single component providing the entire data model object name cannot be found, a message containing the appropriate error is sent back to the requesting consumer component.
- If a single provider component was found, the ADD INSTANCE message is received by the provider. The provider of the data model object checks to confirm that the ADD INSTANCE operation is supported on this data model object. If not supported, a message containing the appropriate error is sent by the provider component and the system routes this error message to the requesting data model consumer component. If the ADD INSTANCE operation is supported by the provider component, the provider adds the requested new instance and sends a confirmation message back to the system containing the new instance number.
- The system routes this confirmation message back to the original requesting data model consumer component.
-

4.4 REMOVE INSTANCE Operation

This operation may be used to remove an existing instance from a supported named multi-instance data model object.

This operation is optional for specific named data model objects. Software components that provide named data model objects may or may not support this operation on each named data model object. Support for specific operations is documented in the component interface definition for each component.

Note: Depending on the component interface definition corresponding to the provider component implementation, the number of instances supported by some multi-instance objects may be static or dynamic. Multi-instance objects with a fixed number of instances are said to be “static” multi-instance objects. The REMOVE INSTANCE operation is not supported for static multi-instance objects. An example of a static multi-instance object is an object that has an instance for each WiFi radio in a device. Since the number of WiFi radios is determined by the hardware implementation, the number of instances cannot be changed by other components dynamically. Multi-instance objects that support REMOVE INSTANCE are said to be “dynamic” multi-instance objects.

The REMOVE INSTANCE operation

- This operation begins when a REMOVE INSTANCE message is sent from a data model consumer component. This message requests that an existing instance be removed from named multi-instance data model object. The message may include either an “alias” name or an instance number corresponding to the instance to be removed
- The system routes the REMOVE INSTANCE message to the single component that provides the object name (if any). If a single component providing the entire data model object name cannot be found, a message containing the appropriate error is sent back to the requesting consumer component.
- If a single provider component was found, the REMOVE INSTANCE message is received by the provider. The provider of the data model object checks to confirm that the REMOVE INSTANCE operation is supported on this data model object and that the corresponding instance exists. If not supported or the instance does not exist, a message containing the appropriate error is sent by the provider component and the system routes this error message to the requesting data model consumer component. If the REMOVE INSTANCE operation is supported by the provider component and the corresponding instance exists, the provider removes the corresponding instance and sends a confirmation message back to the system.
- The system routes this confirmation message back to the original requesting data model consumer component

4.5 SUBSCRIBE ON CHANGE Operation

This operation may be used to request that future event notifications be sent to the requesting component when the value or status of the named data model element / object changes. An event notification may be triggered when the value of a named data model element changes or if the underlying status of the named data model element changes (example an event notification may be generated for the named data model element “Device.Reboot!” when the device reboots.) An event notification may be triggered on a named multi-instance object when an instance is added or removed.

This operation is optional for specific named data model elements / objects. Software components that provide named data model elements / objects may or may not support this operation on each named data model element / object. Support for specific operations is documented in the component interface definition for each component.

The SUBSCRIBE ON CHANGE operation:

- This operation begins when a SUBSCRIBE ON CHANGE message is sent from a data model consumer component. This message identifies the specific name of a data model element / object to be

subscribed to and requests that future event notifications be sent when value or status changes to this name occurs. This message may optionally include notification filter expressions that restrict which changes must generate notifications. Example: A filter expression may be used to specify that a value change must be greater than 10 to generate a notification message.

- The system routes the SUBSCRIBE ON CHANGE message to the single component that provides the data model element / object (if any). If a single component providing the entire data model element / object cannot be found, a message containing the appropriate error is sent back to the requesting consumer component.
- If a single provider component was found, the SUBSCRIBE ON CHANGE message is forwarded to the provider of the data model element / object name. The provider checks to confirm that the SUBSCRIBE ON CHANGE operation is supported on this data model element / object name. If not supported, a message containing the appropriate error is sent by the provider component and the system routes this error message to the requesting data model consumer component. If the SUBSCRIBE ON CHANGE operation is supported by the provider component, the subscription is added and the provider sends a message back to the system to confirm the operation was successful.
- The system routes this confirmation message back to the original requesting data model consumer component.

4.6 SUBSCRIBE ON INTERVAL Operation

This operation may be used to request that future event notifications be sent to the requesting component each time a defined time interval occurs.

This operation is optional for specific named data model elements. Software components that provide named data model elements may or may not support this operation on each named data model element. Support for specific operations is documented in the component interface definition for each component.

The SUBSCRIBE ON INTERVAL operation:

- This operation begins when a SUBSCRIBE ON INTERVAL message is sent from a data model consumer component. This message identifies the name of a data model element and requests that future event notifications be sent on a specified interval value.
- The system routes the SUBSCRIBE ON INTERVAL message to the single component that provides the data model element (if any). If a single component providing the data model element cannot be found, a message containing the appropriate error is sent back to the requesting consumer component.
- If a single provider component was found, the SUBSCRIBE ON INTERVAL message is forwarded to the provider of the data model element name. The provider checks to confirm that the SUBSCRIBE ON INTERVAL operation is supported on this data model element name. If not supported, a message containing the appropriate error is sent by the provider component and the system routes this error message to the requesting data model consumer component. If the SUBSCRIBE ON INTERVAL operation is supported, the subscription is added and the provider sends a message back to the system to confirm the operation was successful.
- The system routes this confirmation message back to the original requesting data model consumer component.

4.7 NOTIFY Operation

This operation may be used to send one or more event notification messages when one or more subscriptions are in effect and the subscribed condition that causes an event occurs.

This operation is optional for specific named data model elements / objects. Software components that provide named data model elements / objects may or may not support this operation on each named data model element / object. Support for specific operations is documented in the component interface definition for each component.

The NOTIFY operation:

- This operation begins when a component that provides events has one or more event subscriptions in effect and a condition occurs that generates an event. The NOTIFY operation supports events from both SUBSCRIBE ON CHANGE subscriptions and SUBSCRIBE ON INTERVAL subscriptions.
- The NOTIFY message is sent from the applicable data model element provider component. This message identifies the name of a data model element that triggered the event notification and includes corresponding data as defined by the Component Interface Definition. This may include previous value and new value (if applicable) and/or other data related to the event. The event notification message is replicated and sent to each consumer component that has an applicable subscription in effect.
- The system routes these event notification messages to each of the applicable subscribing consumer components.

4.8 INVOKE SYNCHRONOUS Operation

This operation may be used to invoke a method that provides a synchronous return response. Multiple input parameters and output parameters may be supported.

This operation is optional for specific named data model elements / objects. Software components that provide named data model elements / objects may or may not support this operation on each named data model element / object. Support for specific operations is documented in the component interface definition for each component.

The INVOKE SYNCHRONOUS operation:

- This operation begins when an INVOKE SYNCHRONOUS message is sent from a data model element consumer component. This message identifies the name of a data model element that provides the method and requests that the method be invoked synchronously using zero or more input parameters and zero or more output parameters.
- The system routes the INVOKE SYNCHRONOUS message to the single component that provides the data model element (if any). If a single component providing the entire data model element/object cannot be found, a message containing the appropriate error is sent back to the requesting consumer component.
- If a single provider component was found, the INVOKE SYNCHRONOUS message is routed to the component that provides the data model element. The provider component checks to confirm that

the INVOKE SYNCHRONOUS operation is supported on this data model element / object. The provider will also confirm that input parameters and output parameters match the definition of this method. If not supported or the input or output parameters do not match what is supported, a message containing the appropriate error is sent by the provider component and the system routes this error message to the requesting data model consumer component. If the INVOKE SYNCHRONOUS operation is supported by the provider component and the input parameters and output parameters match the definition of this method, the provider invokes the method. When the method is finished running it sends a message back to the system to confirm the operation was successful or return an error. Any supported output parameters are also sent in this message.

- The system routes this response message back to the original requesting data model consumer component.

4.9 INVOKE ASYNCHRONOUS Operation

This operation may be used to invoke a method that provides an asynchronous return response. Multiple input parameters and output parameters may be supported.

This operation is optional for specific named data model elements / objects. Software components that provide named data model elements / objects may or may not support this operation on each named data model element / object. Support for specific operations is documented in the component interface definition for each component.

The INVOKE ASYNCHRONOUS operation:

- This operation begins when an INVOKE ASYNCHRONOUS message is sent from a data model element consumer component. This message identifies the name of a data model element that provides the method and requests that the method be invoked asynchronously using zero or more input parameters and zero or more output parameters.
- The system routes the INVOKE ASYNCHRONOUS message to the single component that provides the data model element (if any). If a single component providing the entire data model element/object cannot be found, a message containing the appropriate error is sent back to the requesting consumer component.
- If a single provider component was found, the INVOKE ASYNCHRONOUS message is routed to the component that provides the data model element. The provider component checks to confirm that the INVOKE ASYNCHRONOUS operation is supported on this data model element / object. The provider will also confirm that input parameters and output parameters match the definition of this method. If not supported or the input or output parameters do not match what is supported, a message containing the appropriate error is sent by the provider component and the system routes this error message to the requesting data model consumer component. If the INVOKE ASYNCHRONOUS operation is supported by the provider component and the input parameters and output parameters match the definition of this method, the provider invokes the method. The provider component now sends a message back to the original consumer component that the INVOKE ASYNCHONOUS message for the data element / object was received and is being processed.

- When the method is finished processing it sends a message back to the system containing any supported output parameters.
- The system routes these response message(s) back to the original requesting data model consumer component.

5. Data Model Structure

The data model naming convention uses both “elements” and “objects” to create its structure. The terms “element” and “object” are defined in section 3 of this document.

The specific list of the objects and elements supported in each software component is provided in the Component Interface Definition for each version of that component. Where applicable, the Feature Flags required to enable specific objects / elements are also documented in the Component Interface Definition.

OBJECT NAMES:

The data model naming convention is hierarchical. Its top level and intermediate levels are composed of object names separated by the dot “.” character.

A few examples of how the data model hierarchy uses objects are:

- ”Device.”
- “Device.WiFi.”
- “Device.WiFi.DataElements.”
- “Device.WiFi.DataElements.Network

Clarification Note: Individual object names (like “WiFi”) are referred to as an object and the entire path (like Device.WiFi.DataElements.Network) can both be referred to as an object.

Some object names are “multi-instance” objects. These are like an array. An example is:

- Device.WiFi.Radio.{i}. // this is used to describe an array of WiFi Radios.

The use of the letter “i” enclosed by parentheses “{i}” is used to indicate all currently existing instances. The count of the currently existing instances is provided by a data element using the *NumberOfEntries label. This is always located in the object hierarchy directly above the multi-instance object.
For example, if there are 3 current instances of the Device.WiFi.Radio.{i} object then a GET operation on Device.WiFi.RadioNumberOfEntries element will return a value of 3.

Each specific instance in a multi-instance object may be further identified by using either an instance “number” or an instance “alias” name. An alias name is enclosed with square brackets [ ]. A few examples are:

- Device.WiFi.Radio.2. // sub-objects and elements in this tree all refer to WiFi radio 2
- Device.WiFi.Radio.[FiveGHz]. // sub-objects and elements in this tree all refer to the FiveGHz radio

Suppose, Device.WiFi.Radio.2.Alias = FiveGHz // the alias name of radio instance 2 is “FiveGHz”.
Then, the object name “Device.WiFi.Radio.[FiveGHz].” is equivalent to the object name “WiFi.Radio.2.”.

ELEMENT NAMES:

The lowest levels of the data model hierarchy are composed of individual element names. A complete path to a data model element is formed by a combination of object name(s) and an element name.

A few examples include:

```
Device.WiFi.Radio.[FiveGHz].Enable                          // used to enable/disable the radio
Device.WiFi.Radio.[FiveGHz].CurrentOperatingChannelBandwidth // contains channel bandwidth value
Device.WiFi.Radio.2.Stats.BytesSent                         // contains the current byte sent count
Device.WiFi.Radio.[FiveGHz].Stats.BytesReceived             // contains current byte received count
Device.WiFi.Radio.[FiveGHz].Stats.Reset()                   // a method used to reset the counters
```

NON-STANDARD NAMES

Data Model object and element names should follow standard TR-181 specifications. When the TR-181 spec does not cover specific requirements, non-standard data model object and element names may be created. These non-standard names must use a special prefix flag indicating that these are non-standard. This prefix flag must be at the highest (left most) non-standard object or element location in the path. Non-standard data models objects and elements specific to RDK should use the “X_RDK_” flag. Older data models specific to RDK may continue to use the “X_RDKCENTRAL-COM_” flag but new data models should use the shorter “X_RDK_” flag.

An example of a non-standard object name is:
Device.WiFi.X_RDK_MyNewObject.ElementName

An example of a non-standard element name is:
Device.WiFi.X_RDK_MyNewElementName

DATA MODEL OPERATIONS:

Data model objects and elements may support one or more operations defined in section 4 of this document. The lists of supported data model objects / elements and their respective supported operations are documented in the *Component Interface Definition* for each applicable component. This Component Interface Definition is specific to a component and versioned. Each version of the document is associated with one or more versions of the software component. Where applicable, the definition of each data model Object and Element may also be associated with one or more build-time feature flags used by component.

For example, the following operations may be supported on each of the data model elements below:

```
Device.WiFi.Radio.[FiveGHz].Enable                          // GET, SET, SUBSCRIBE-ON-CHANGE
Device.WiFi.Radio.[FiveGHz].CurrentOperatingChannelBandwidth // GET, SUBSCRIBE-ON-CHANGE
Device.WiFi.Radio.2.Stats.BytesSent                         // GET, SUBSCRIBE-ON-INTERVAL
Device.WiFi.Radio.[FiveGHz].Stats.BytesReceived             // GET, SUBSCRIBE-ON-INTERVAL
Device.WiFi.Radio.[FiveGHz].Stats.Reset()                   // INVOKE-SYNCHRONOUS
```

Parameter value types and descriptions:

If the data model element supports a parameter value used for GET, SET and/or SUBSCRIBE operations or supports input/output parameters used for methods, then the Component Interface Definition for this element must include a parameter type description. The definition must also describe the usage or meaning of the value.

For example:

```
Device.WiFi.Radio.[FiveGHz].Enable          // Type:  Boolean
                                            // Usage: 0 = disable, 1 = enable

Device.WiFi.Radio.[FiveGHz].CurrentOperatingChannelBandwidth   // Type:  String
                                    // Usage: The string value is an enumeration of supported values including
                                    // 20MHz, 40MHz, 80MHz, 160MHz, 80+80MHz, 320MHz-1, 320MHz-2

Device.WiFi.Radio.[FiveGHz].Stats.BytesReceived  // Type:  Unsigned Long
                                                 // Usage: The cumulative count of bytes received on this
                                                 // radio since the counter was reset. Note: The value may
                                                 // wrap when the count exceeds the max value supported
                                                 // by unsigned long
```

Some elements / objects support SUBSCRIBE operations. After a valid SUBSCRIBE operation, the element or object responds with an event notification message when an event notification is triggered. The event notification may include previous and current values and/or other property values related to the event.
For example, if a component has previously sent a SUBSCRIBE-ON-CHANGE message for the data model element “Device.WiFi.Radio.[FiveGHz].CurrentOperatingChannelBandwidth”, an event notification operation will be triggered each time the value changes.

Some data model elements support SUBSCRIBE operations and do NOT support GET or SET operations. The names for these elements may optionally use a “!” character at the end of the element name to indicate this this data model element supports event subscriptions but does not support GET or SET.
An example is:

```
Device.Boot!                // An event used to indicate that the device was reset
Device.DeviceInfo.MemoryStatus.MemoryCriticalState!  // an event indicating a critical condition is reached.
```

Event notifications may contain related information parameters. For example, the TR-181 standard definition for the event notification for Device.DeviceInfo.MemoryStatus.MemoryCriticalState! includes the following property value that is sent with the event notification:

```
“MemUtilization”        //  Type:  unsigned int (:100)
                        // Memory utilization, in percent, rounded to the nearest whole percent
```

Operation on object names:

The operations described in section 4 of this document can also be used on object names. In this case the component providing this object will respond as if the operation was performed on all supported elements under that object name. Note: Operations are not supported for an object name that spans multiple Component Interface Definitions. In this case operations should be applied to a lower level object name provided by a single Component Interface Definition.

For example: If the object name “Device.WiFi.Radio.[FiveGHz].Stats.” is contained within a single Component Interface Definition and supports GET operations on at least one element under this object, A GET of this object name will respond with a list of all elements under this object that support GET and their respective values. Any elements under the object “Stats” that do not support GET will not be included in the response.

6. Data Model Documentation Template

This document defines a Data Model Documentation Template used to describe the json format used for the Component Interface Definition documents. Each of these documents all the data model objects and elements in a software component interface. This template defines a json based approach. This approach supports future dashboard tooling that can consume these documents and present various filtered and summary views. These views may present combinations of multiple Component Interface Definitions used to describe the data model objects/elements supported in a custom software build. This tooling may also present summary views that include all or a filtered subset of Component Interface Definitions and include filtering by document version(s) and selected feature flags.

```
{
   "componentInterfaceDefinition": {
      "name": "Add the component name that implements this interface here",
      "version": "specify the document version here",  // use major, minor, revision format
      "description": "Add a description of this component interface here"
   }

   "object": [                  // add a high-level object provided by this component interface here
   {
      "path": "Add the complete path here",                    // example:  Device.WiFi.
      "name": "add the new object name here",     // example:  WiFi.
      “description”: “add the object’s description here”,

      // work your way down the hierarchy
      // add elements directly under the high level object
      // add any objects directly under the high level object
      // after each object, define the next lower level under that object
      // continue recursively until everything under this object is defined.
      // then repeat for other objects
```

```
    “element”: [          // add element names at this level, this is an example that supports GET and Subscribe
      {
        “path”: “Add the path here”,    // example: Device.WiFi.
        “name”: “add the element name here”, //example: RadioNumberOfEntries
        “access”: [“read”,”subscribeOnChange”],  // list all access supported by this element
        “type”: “unsignedInt”, // specify the data type here
        “description”: “Add a description here including how the element is used”,
        “persistence”:  “yes”,  // Is this element’s value persisted across normal reboot?  “yes” or “no”
        “factoryDefault”: “value”, // specify the factory reboot default value, use empty string if n/a
      },
    ],
    “element”: [          // this is an example of a method name
      {
        “path”: “Add the higher-level path here”,          // example: Device.WiFi.
        “name”: “add the element name here”,             //example: Reset()
        “access”: [“invokeSync”],  // list all access supported by this element
        “description”: “Add a description here including what this does and how it is used”,
         // example description is “This method is a request to reset the WiFi subsystem
         // without resetting the device”
        “input”: [        // add input names, types and descriptions
            [“name”:”add name here”, “type”:”add type here”, “description”:”add description”],
            [“name”:”add name here”, “type”:”add type here”, “description”:”add description”],
            [“name”:”add name here”, “type”:”add type here”, “description”:”add description”],
        ],
        “output”: [      // add output names, types and descriptions
            [“name”:”add name here”, “type”:”add type here”, “description”:”add description”],
            [“name”:”add name here”, “type”:”add type here”, “description”:”add description”],
            [“name”:”add name here”, “type”:”add type here”, “description”:”add description”],
        ],
      },
    ],
    "object": [     // add a next level sub-object provided by this component interface here
      {
        "path": "RootObject.object.{i}.object.{i}.object",             // example:  Device.WiFi.Stats.
        "name": "add the new object name here",         // example:  Stats.
        “description”: “add the object’s description here”,

        // add all elements and object names under this object
        // continue recursively until every level under this object is defined
       },
    ],
  },
}
```

7. Interface Inventory and API Coverage Report

This section lists the Component Interface Definitions for RDKB. Each Component Interface Definitions lists and defines the data model objects and elements supported in each version of each RDKB core component. Where applicable, the Feature Flags required to enable specific objects / elements are also documented in each Component Interface Definition.

The following table provides the Interface Inventory and the current API Coverage Report for the list of Component Interface Definition documents.

Interface Inventory	API Coverage Report
WebUI	Document not available
WebUI BWG	Document not available
USP PA	Document not available
Web PA	Document not available
tr069-protocol-agent	Document not available
WebConfig	Document not available
dcm agent	Document not available
xConf client	Document not available
RFC	Document not available
RDK OVS Agent	Document not available
Ethernet Agent	Document not available
OneWiFi	Document not available
CCSP WiFi	Document not available
Mesh Agent	Document not available
ieee1905	Document not available
moca-agent	Document not available
lan-manager-lite	Document not available
cable-modem-agent	Document not available
xDSL Manager	Document not available
EPON Agent	Document not available
gpon-manager	Document not available
PPP Manager	Document not available
cellular-manager	Document not available
Cellular Modem Mgr	Document not available
hotspot	Document not available
dhcp-manager	Document not available
P&M	Document not available
WAN Manager	Document not available

core-net-library	Document not available
secure-upnp	Document not available
ccsp-xdns	Document not available
advanced-security	Document not available
Utopia	Document not available
component-registry	Document not available
miscellaneous-broadband	Document not available
led-manager	Document not available
platform-manager	Document not available
json-hal-lib	Document not available
sysint-broadband	Document not available
common utilities	Document not available
MTA Agent	Document not available
power manager	Document not available
telco-voice-manager	Document not available
hal-voice-asterisk	Document not available
rdk_logger	Document not available
syslog_helper	Document not available
crashupload	Document not available
remote_debugger	Document not available
T2 Telemetry	Document not available
Test & Diagnostics	Document not available
ipoe-health-check	Document not available
cpuprocanalyzer	Document not available
Break pad	Document not available
sys_mon_tools	Document not available
harvester	Document not available
RBUS	Document not available
inter-device-manager	Document not available
common-library	Document not available
DMCLI	Document not available
notify-component	Document not available
PSM	Document not available
DSM	Document not available
Dobby	Document not available
Barton Core	Document not available
home-security	Document not available
rdk-cert-config	Document not available
rdkssa	Document not available
rdk-libunpriv	Document not available
libsyscallwrapper	Document not available
rdkb-apparmor-profiles	Document not available